

REFLECTIONS

TASK 1:

Config (Factor III): Why .env is better than hardcoding

Hardcoding passwords or API keys is a major security risk because once code is committed to version control (like Git), those secrets are visible to anyone with access and are nearly impossible to erase from history.

- Using .env files keeps sensitive data out of your repository.
- Additionally, it allows you to change settings for different environments (like moving from a local test database to a production one) without ever touching the actual source code.

Backing Services (Factor IV): Why separate databases are useful

Treating a database as a "backing service" means viewing it as an attached resource rather than a hardwired part of the app. This loose coupling makes your application modular and "swappable." If you need to switch from a local database to a cloud provider like Amazon RDS, or if a primary server fails and you need to attach a backup, you can do so by simply updating a URL in your configuration—zero code changes required.

Dev/Prod Parity (Factor X): Why Docker ensures similarity

Docker creates a consistent "container" that includes the exact same operating system, libraries, and dependencies regardless of where it runs. By using the same Docker image in development and production, you eliminate the "it works on my machine" syndrome. It ensures that if the code passes tests on a developer's laptop, it will behave identically in the production environment.

TASK 2:

Reflection (The "Why")

In professional software, we follow the Twelve-Factor App methodology. Here is why we are doing things this way:

Factor II: Dependencies

Question: How do you fix library versions?

Answer: We use a requirements.txt file and a Virtual Environment. This ensures that if we share our code, the other person uses the exact same version of FastAPI as you. It prevents the "It works on my machine" excuse.

By using a requirements.txt and a virtual environment, I ensure that the application has the exact same version of FastAPI and SQLAlchemy across all environments. This eliminates "environment drift" and ensures the API is reproducible.

Factor IV: Backing Services

Question: How can you change PostgreSQL to another database later?

Answer: Because we use SQLAlchemy (an ORM), our Python code doesn't care which database is behind it. By changing only one line in our configuration, we could switch from PostgreSQL to MySQL or SQLite without rewriting our logic.

Using SQLAlchemy (ORM) allows me to interact with the database via Python objects. If we moved from PostgreSQL to a different service like MySQL, I would only need to update the DATABASE_URL in the .env file; the CRUD logic would remain untouched.

Factor III: Config Management

Question: Why keep config in .env?

Answer: Security and flexibility.

We never want to upload your database password to GitHub. By using a .env file, we can have one password for our laptop (development) and a different, stronger password for the real website (production) without changing the code itself.

I have kept the database credentials in the .env file. This allows me to use local credentials for development while using more secure, encrypted secrets in a production deployment without modifying the source code.

TASK 3:

Final Reflection (200 words)

Factor VIII (Concurrency):

In this task, I implemented Factor VIII by moving from sequential processing to concurrent processing using asyncio. Sequential processing acts like a single cashier; it forces each database query to wait for the previous one to finish, creating a bottleneck. By using asyncio.gather, I treated the API like a bank with multiple cashiers, allowing the system to handle all 8 table counts simultaneously. This significantly reduces the total response time for the dashboard.

Furthermore, by combining this with Factor II (Dependencies) and Factor IV (Backing Services), the application remains scalable. If the database grows or we switch from PostgreSQL to another service, our modular CRUD functions and concurrent architecture ensure that the API remains high-performing without needing a total rewrite. This setup demonstrates how modern cloud-native applications handle high loads by efficiently utilizing system resources rather than staying idle while waiting for external database responses.